YORK UNIVERSITY

Lassonde School of Engineering
Department of Electrical Engineering and Computer Science

EECS 3311: Software Design

Fall 2026 — Course Project Stage 1 Report
Architecture, Design, and Traceability of AI Agent System

Author:
Abhishek Saraff
220434296

Instructor: Song Wang

Date: October 10, 2026

Project Title: FleetOps (Heterogeneous Multi-Agent Robotics System — H-MARS)

Contents

1 Project Overview **2**
1.1 System Name and Codename . . . 2
1.2 1. What Problem Does Your Project Solve? . . . 2
1.3 2. Who Are the Target Users? . . . 2
1.4 3. What Can the Agent Do? . . . 2
1.5 4. Why is an AI Agent Appropriate for This Problem? . . . 3
1.6 5. Which AI and LLM Models Do You Plan to Use? . . . 3
1.7 6. How Will the AI Model Interact with the Rest of the Software System? . . . 3

2 Detailed Feature Specifications **5**

3 UML Class Diagram and Design Patterns **13**
3.1 Design Pattern Explanations . . . 13
3.1.1 1. Command Pattern . . . 13
3.1.2 2. Strategy Pattern . . . 13
3.1.3 3. Composite Pattern . . . 14
3.1.4 4. Observer Pattern . . . 14
3.1.5 5. Decorator and Safety Gate Pattern . . . 14
3.1.6 6. Facade Pattern . . . 15
3.1.7 7. Adapter Pattern . . . 15
3.1.8 8. Factory Method Pattern . . . 16

4 Use-Case Diagram and Descriptions **17**
4.1 Detailed Use-Case Descriptions . . . 19

5 Sequence Diagrams **22**

6 Feature-to-Design Traceability Table **24**

7 Feature Realization Explanations **25**

1 Project Overview

1.1 System Name and Codename

FleetOps (Heterogeneous Multi-Agent Robotics System — H-MARS)
An Autonomous LLM-Driven Task Allocation and Verification Engine for Heterogeneous Robotic Fleets.

1.2 1. What Problem Does Your Project Solve?

Coordinating heterogeneous autonomous teams combining aerial reconnaissance drones, tracked ground vehicles, quadruped leg-robots, and heavy-lift utility helicopters presents a high-dimensional command problem:

1. **Unstructured Mission Directives:** Translating real-time open intelligence, operator text commands, or multi-source event feeds into valid, coordinate-based multi-vehicle flight and path plans is prone to human oversight under time-sensitive pressure.
2. **Domain-Specific Kinematic Constraints:** Heterogeneous vehicles operate under strict, asymmetric physical rules. For example, aerial drones scan fast but have constrained flight battery, ground rovers carry payloads but cannot breach high-elevation debris, robodogs cover uneven radius paths, and helicopters require cleared landing zones.
3. **Safety and Non-Determinism Risks:** Directly executing natural-language LLM plans in real hardware or simulated environments risks catastrophic hallucinations, collisions, or out-of-bounds pathing.

FleetOps solves this by providing a unified LLM-driven Mission Control Engine that automatically ingests unstructured field feeds, uses AI tool calling to query persistent database state, generates multi-vehicle task queues, and enforces a deterministic Java safety gate (`ActionVerifier`) before any action touches the 2D execution grid.

1.3 2. Who Are the Target Users?

- **Robotics Mission Control Engineers:** Tactical operators managing heterogeneous autonomous hardware who need an intelligent high-level orchestrator to auto-delegate tasks across mixed fleets.
- **Autonomous Systems and Swarm Researchers:** Engineers evaluating non-deterministic LLM task allocation algorithms against strict deterministic physical rule checkers.
- **Defense and Field Security Operators:** Incident commanders who require a dual-mode system supporting both autonomous artificial intelligence and manual override controls on a dashboard to monitor 2D spatial deployment in real time.

1.4 3. What Can the Agent Do?

The autonomous AI agent serves as a high-level Strategic Fleet Orchestrator capable of reasoning, database tool querying, structured task allocation, and performance benchmarking:

- **Ingest and Parse Intelligence Feeds:** Continuously streams external open intelligence feeds from Telegram or RSS sources, parses unstructured event text into structured location and hazard objects, and invokes database tools to eliminate redundant mission logs.

- **Decompose and Allocate Heterogeneous Tasks:** Analyzes complex field objectives and constructs multi-stage asset schedules such as dispatching aerial reconnaissance drones to map ground paths, routing tracked rovers along cleared grid vectors, deploying quadruped robodogs for localized perimeter radius scans, and signaling helicopters for payload extractions.
- **Execute Multi-Tool Database Operations:** Uses LLM tool calling to query local state including vehicle battery levels, current payload capacity, and spatial bounds before proposing execution steps.
- **Monitor and Adapt Real-Time Execution:** Tracks live simulation feedback loops from the 2D grid matrix and dynamically triggers re-planning if ground obstacles or low-battery thresholds are encountered.
- **Synthesize Post-Mission Analytical Reports:** Generates post-flight diagnostic reports comparing AI-discovered target data against ground-truth simulation entities to evaluate tactical efficiency.

1.5 4. Why is an AI Agent Appropriate for This Problem?

Traditional software relies on hardcoded procedural scripts or static optimization algorithms that lack the contextual flexibility required to parse natural-language field intelligence or adaptively orchestrate distinct asset classes under shifting constraints.

However, pure LLM generation cannot be trusted for direct physical execution due to structural hallucinations. An AI Agent framework with tool calling provides the exact bridge needed: Gemini delivers natural-language reasoning, tactical task decomposition, and feed parsing, while handing off kinematic validation and state tracking to deterministic Java verification classes.

1.6 5. Which AI and LLM Models Do You Plan to Use?

- **Primary LLM:** Google Gemini accessed via the Java SDK and REST API, acting as the primary reasoning engine for feed ingestion, tool selection, plan generation, and report synthesis.
- **Edge and Offline CLI Fallback:** A local lightweight model interface such as Gemma accessible via command-line execution when remote cloud APIs are offline.

1.7 6. How Will the AI Model Interact with the Rest of the Software System?

The AI model operates inside a fully decoupled Agent Orchestrator Pipeline designed with strict safety boundaries:

1. **Ingestion and Database Tool Call Loop:** Unstructured reports enter via the adapter feed layer. Gemini processes the text and invokes database tool calls to check if the target location or objective has already been logged.
2. **Task Queue Generation:** Gemini produces a structured JSON array of concrete `Command` objects including `ScanPathCommand`, `TraverseRoverCommand`, `DeployRobodogCommand`, and `HelicopterDispatchCommand`.
3. **Deterministic ActionVerifier Gate:** Proposed commands pass through an `ActionVerifier` Java class. This safety gate checks hard physical constraints such as whether a specific grid cell contains an impassable obstacle or if a drone maintains sufficient battery. Invalid commands are blocked before execution.

4. **Dual Execution Engine (GUI and CLI):** Approved commands populate the GUI action queue or command-line queue. The operator can accept all actions or manually override individual commands.

5. **Real-Time 2D Render and State Observer:** As commands execute on the 2D grid matrix, the observer pattern updates robot positions, battery gauges, and target markers in real time.

2 Detailed Feature Specifications

1. **F01 — Intelligence Feed Ingestion and Parsing**

   - **Description:** Ingests raw, unstructured incident text from external streams including Telegram Channel Bot APIs, open RSS news feeds, or manual operator text pastes, and parses them into structured mission target objects.
   - **User Interaction:** The user accesses the Intelligence Stream tab in the GUI, clicks the Scan Feeds button, or pastes raw text into an input box and submits. Command-line access is available via `fleetops scan -source telegram`.
   - **Input:** Raw text or JSON string payloads from the Telegram API, RSS feeds, or manual text input boxes.
   - **Output:** Structured JSON payload representing detected mission targets including coordinates, hazard severity, and survivor counts.
   - **AI Involvement:** AI-Based, utilizing the Gemini LLM to parse raw natural-language strings into structured data.
   - **Expected Workflow:**
     - User triggers scan operation.
     - System fetches raw feed data.
     - Data is sent to the Gemini LLM.
     - The LLM extracts entities and outputs a structured JSON payload.
     - Extracted targets populate the pending intelligence table in the GUI and command-line interface.
   - **Error and Alternative Cases:** API connection timeouts or unparseable unstructured text trigger an exception catch. The system displays an alert dialog in the GUI, logs the error, and falls back to a manual text entry prompt.

2. **F02 — Vector and Database Deduplication Tool**

   - **Description:** Prevents duplicate mission task creation by querying a local SQLite or H2 database via an LLM tool call to verify whether an incoming report matches an existing logged event.
   - **User Interaction:** Executed automatically during feed parsing, or manually triggered by selecting a pending report and clicking Verify Against Database. Command-line access is available via `fleetops verify-db -id <target_id>`.
   - **Input:** Parsed mission target object containing geographic location, event timestamp, and incident description.
   - **Output:** Boolean duplicate indicator and a similarity match score object from the database.
   - **AI Involvement:** Hybrid, where Gemini executes an automated tool call to query the database while the Java persistence layer runs SQL and vector similarity queries deterministically.
   - **Expected Workflow:**
     - Gemini evaluates an extracted target.
     - The model invokes `queryDatabaseTool(target_coordinates)`.
     - The Java database engine searches for existing entries within a defined spatial radius.
     - Results are returned to Gemini.

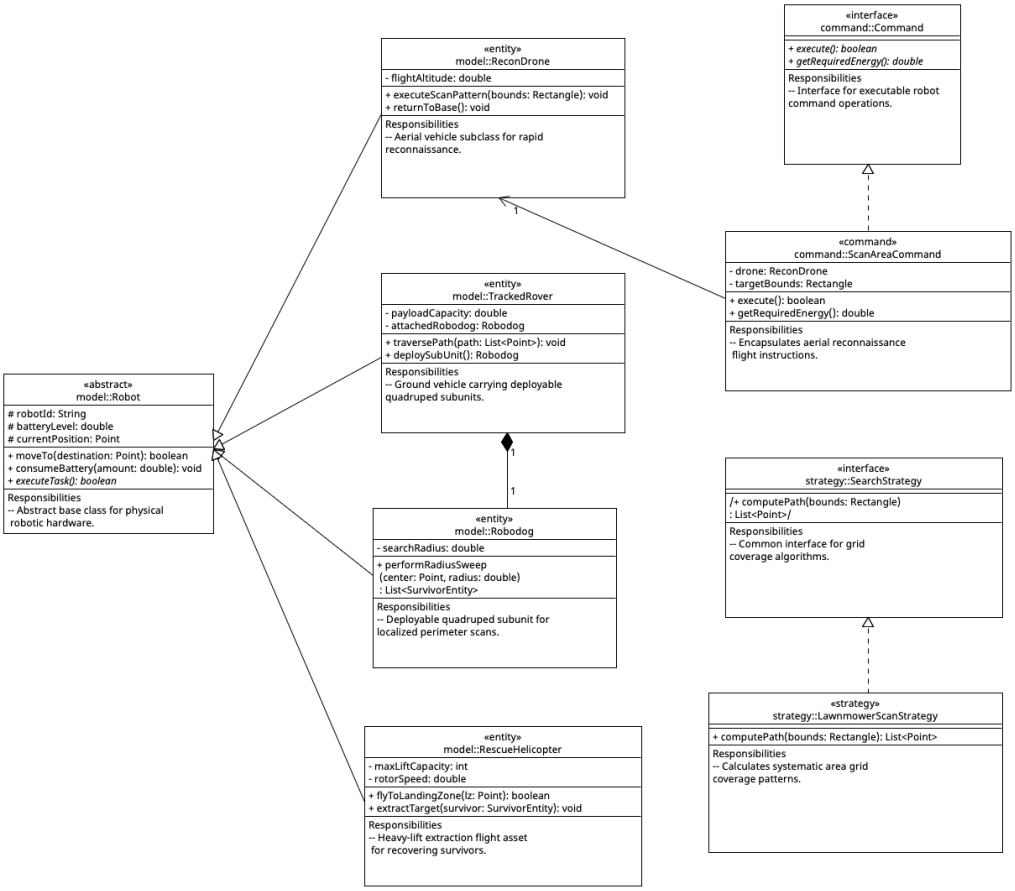


Figure 1: Core Domain and Robot Hierarchy Architecture

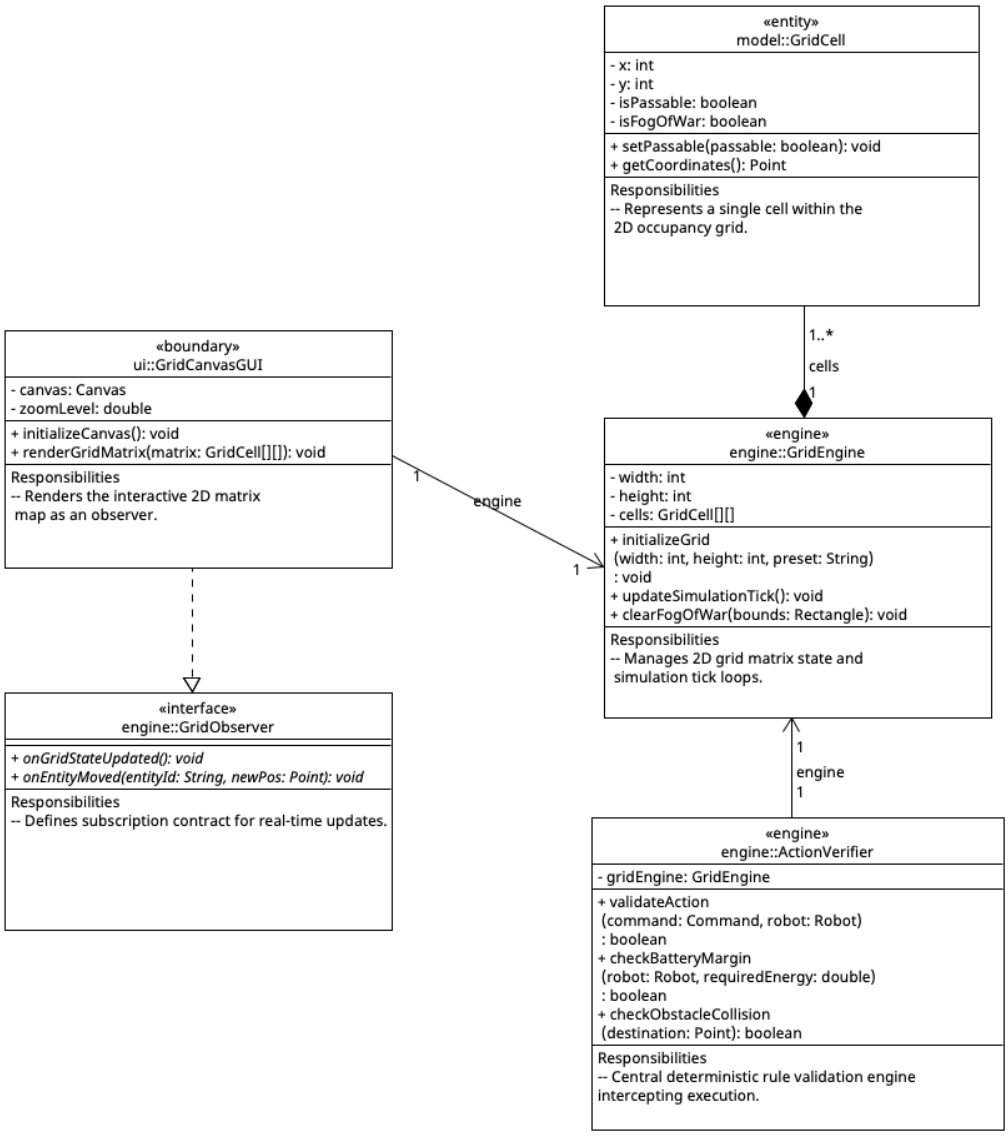


Figure 2: Simulation Engine and Safety Gate

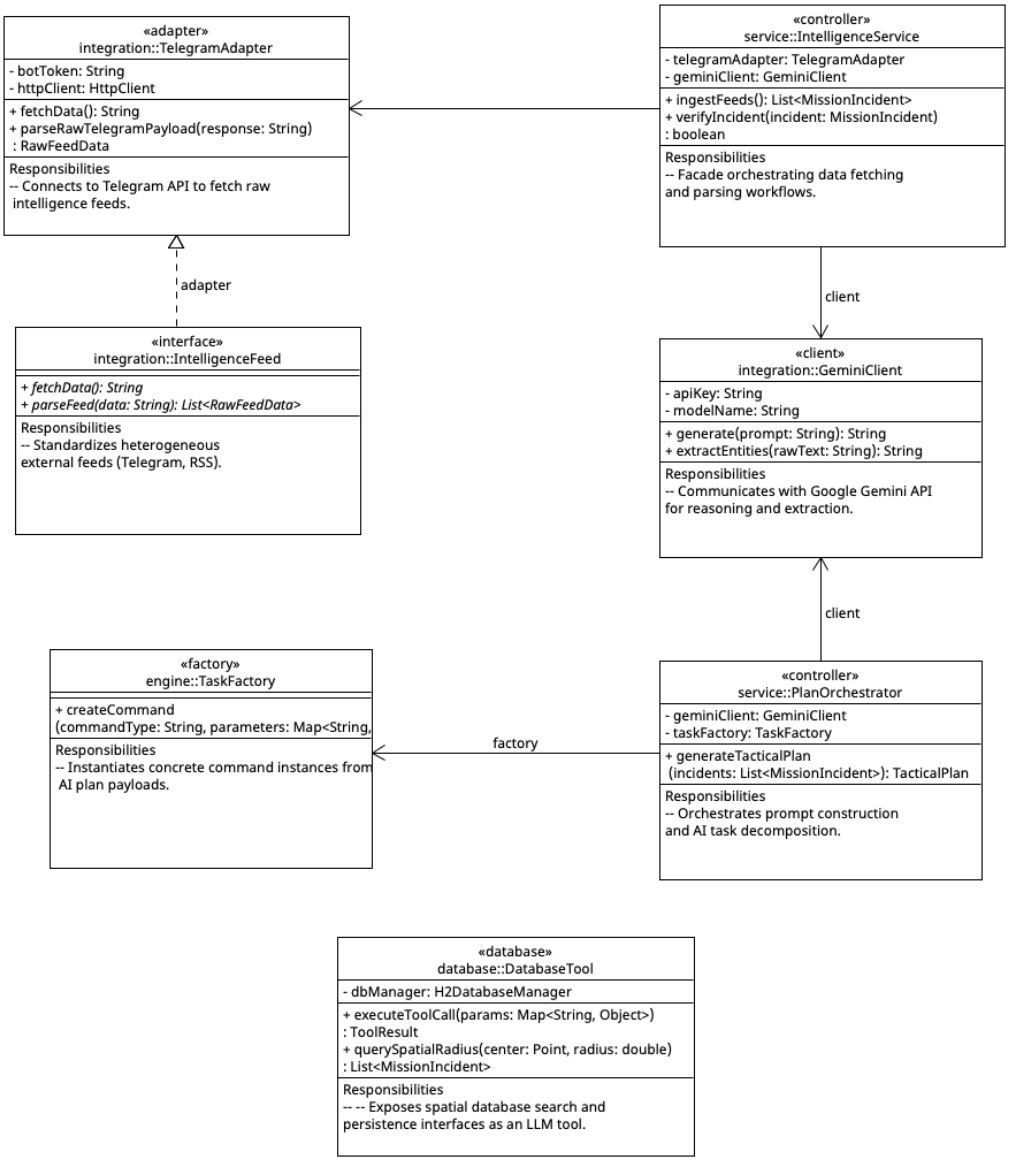


Figure 3: AI Orchestration and Integration Component

  - If determined to be a duplicate, the report is merged or flagged as resolved.
- **Error and Alternative Cases:** Database locking or corruption exceptions are caught by the system, which flags the item as unverified due to database unavailability and permits manual operator approval.

3. **F03 — Autonomous Heterogeneous Task Plan Generation**

- **Description:** Analyzes verified mission targets and automatically generates a multi-robot tactical plan assigning specialized roles across aerial drones, tracked rovers, robodogs, and helicopters.
- **User Interaction:** User selects target incidents from the workspace and clicks the Generate Tactical Plan button. Command-line access is available via `fleetops plan -target <id>`.
- **Input:** Verified mission target list and current available robot fleet status including battery, capacity, and location.
- **Output:** Structured multi-step task plan consisting of sequential command objects queued for execution.
- **AI Involvement:** AI-Based, utilizing Gemini reasoning loops to decompose complex field objectives into robot-specific subtasks.
- **Expected Workflow:**
  - User clicks generate.
  - System packages target data and fleet state into a prompt.
  - Gemini synthesizes the task sequence.
  - The task plan populates the Plan Approval GUI tab with estimated completion times and assigned robot IDs.
- **Error and Alternative Cases:** If no available robots meet mission requirements due to low battery, Gemini outputs a fleet constrained warning and generates a degraded contingency plan or prompts the user to recharge assets.

4. **F04 — Interactive Task Queue Approval and Manual Override**

- **Description:** Displays proposed AI task sequences in a dedicated GUI tab, allowing the operator to accept all actions, reject specific tasks, or manually insert and reorder robot commands.
- **User Interaction:** User opens the Plan Approval tab, clicks checkmark or cross icons next to individual tasks, or clicks Approve and Queue All. Command-line access is available via `fleetops approve -plan-id <id> -tasks all`.
- **Input:** Proposed task list from feature F03 and manual user modifications such as task toggles and reordering.
- **Output:** Finalized execution queue committed to the 2D simulation loop.
- **AI Involvement:** Deterministic, using pure UI and command-line state management logic with user-in-the-loop validation.
- **Expected Workflow:**
  - Proposed plan appears in the table.
  - User reviews assignments.
  - User toggles off unwanted actions or edits targets.
  - User clicks Commit Plan to send commands to the simulation execution queue.

- **Error and Alternative Cases:** If a user attempts to commit an empty plan, the GUI highlights the error, disables the commit button, and displays a tooltip requiring at least one valid command.

5. **F05 — 2D Environment and Dynamic Entity Generator**

- **Description:** Renders and manages the 2D occupancy grid matrix, instantiating terrain obstacles, hazard zones, and survivor entities with distinct behavior classes including stationary, drift, and high movement.
- **User Interaction:** User opens the 2D Grid View tab, selects terrain presets from a dropdown, or uses grid tools to manually click and place obstacles and survivors. Command-line access is available via `fleetops grid init -size 20x20 -preset flood`.
- **Input:** Grid dimension specifications, terrain configuration files, or manual mouse-click coordinates.
- **Output:** Visual 2D grid matrix canvas with colored cells representing terrain, obstacles, survivors, and home bases.
- **AI Involvement:** Deterministic, operating via physics and state loops for grid logic and dynamic survivor movement vectors.
- **Expected Workflow:**
  - Grid initializes.
  - Survivor entities instantiate with movement properties.
  - The game loop updates entity positions per tick.
  - The view updates the canvas at 30 frames per second.
- **Error and Alternative Cases:** Invalid grid bounds or entity placement over impassable wall cells causes the system to automatically reseat the entity to the nearest open grid cell and log a warning.

6. **F06 — Aerial Reconnaissance and Path Mapping**

- **Description:** Commands reconnaissance drones to execute grid-coverage flight patterns, revealing unscanned cells and clearing safe ground navigation vectors for rovers.
- **User Interaction:** User accepts an AI drone task or clicks Manual Drone Scan, selecting a target grid area on the 2D map canvas. Command-line access is available via `fleetops dispatch -unit DRONE_1 -action scan -bounds 0,0,5,5`.
- **Input:** Drone ID, target cell coordinates, and current battery level.
- **Output:** Updated grid state with revealed cells and cleared terrain, along with recalculated optimal ground paths.
- **AI Involvement:** Hybrid, combining AI-proposed target area coverage with deterministic pathfinding algorithms.
- **Expected Workflow:**
  - Command is triggered.
  - Drone moves cell by cell.
  - Unscanned fog of war is removed.
  - Obstacles are marked on the global map and pathfinding meshes are updated.
- **Error and Alternative Cases:** If drone battery drops below fifteen percent mid-flight, the system aborts the scan, automatically issues a return to base command, and marks unvisited cells as unscanned.

7. **F07 — Ground Traversal and Perimeter Radius Sweep**
   - **Description:** Navigates tracked rovers along paths cleared by feature F06 to waypoints, where quadruped robodogs deploy to perform circular radius sweeps detecting survivors.
   - **User Interaction:** User clicks Execute Ground Sweep or selects a robodog on the grid to manually draw a search radius. Command-line access is available via `fleetops dispatch -unit ROVER_1 -action sweep -radius 3`.
   - **Input:** Rover path vector, robodog deployment center point, and search radius.
   - **Output:** Detected survivor locations flagged on the map and logged survivor movement profiles.
   - **AI Involvement:** Hybrid, using AI to optimize search radius based on report severity and geometric circle search algorithms executing on the grid.
   - **Expected Workflow:**
     - Rover travels along the path.
     - Reaches the designated waypoint.
     - Deploys the robodog.
     - Robodog scans cells within the specified radius and flags found survivors.
   - **Error and Alternative Cases:** If a rover encounters an unexpected dynamic obstacle blocking the cleared path, it halts, flags the path as blocked, and requests route recalculation.

8. **F08 — Helicopter Extraction Dispatch**
   - **Description:** Dispatches heavy-lift utility helicopters to confirmed survivor locations to perform aerial extractions and transport victims to base.
   - **User Interaction:** Automatically triggered upon survivor verification or manually invoked by clicking a flagged survivor icon on the map and selecting Dispatch Extraction. Command-line access is available via `fleetops dispatch -unit HELI_1 -target-survivor S_01`.
   - **Input:** Survivor grid coordinates, helicopter availability, and landing zone safety clearance.
   - **Output:** Survivor status updated to extracted, rescue counter incremented, and helicopter returned to base.
   - **AI Involvement:** Deterministic and hybrid, combining deterministic flight path execution with AI options to prioritize extractions based on survivor movement types.
   - **Expected Workflow:**
     - Survivor is flagged.
     - Helicopter departs base directly to the target.
     - Hover and pickup animation completes.
     - Survivor is removed from active grid and metrics are updated.
   - **Error and Alternative Cases:** If a landing zone is blocked by high-elevation obstacles, the helicopter hovers at a safe distance, flags the landing zone as invalid, and waits for ground units to clear or user override.

9. **F09 — Deterministic ActionVerifier Safety Gate**
   - **Description:** Intercepts every proposed robot action from both AI and users to mathematically validate physical feasibility, battery margins, wall collisions, and map boundaries before execution.

- **User Interaction:** Operates silently in the background, displaying an error modal or console alert if a command is blocked for safety violations with the message `[REJECTED]: ActionVerifier blocked task`.
- **Input:** Proposed command object, target robot hardware specifications, and current 2D grid matrix state.
- **Output:** Validation decision yielding accepted or rejected status with a specific failure reason code.
- **AI Involvement:** Deterministic, operating as a pure Java rule engine acting as an immutable safety filter.
- **Expected Workflow:**
  - Command is submitted.
  - ActionVerifier checks boundary bounds, collision matrices, and battery drain formulas.
  - If valid, the command passes to the simulation thread.
  - If invalid, the command is rejected.
- **Error and Alternative Cases:** If AI proposes a command moving a ground rover into a deep water cell, ActionVerifier intercepts, drops the command, and prompts the AI to re-plan.

10. **F10 — Post-Mission Ground-Truth Analytics and Reporting**

- **Related Use Case:** UC10 — Generate Analytics Report
- **Related Sequence Diagram:** SD10 — Analytics and Synthesis
- **Classes Involved:**
  - `AnalyticsGUI`: Displays final mission charts and text reports.
  - `ReportGenerator`: Compiles performance stats and invokes summary templates.
  - `GeminiClient`: Synthesizes qualitative executive performance summaries.
  - `GridEngine`: Provides ground-truth benchmark data.
- **Important Methods:**
  - `ReportGenerator.compileMetrics()`
  - `ReportGenerator.generateReport()`
  - `GeminiClient.synthesizeExecutiveSummary()`
  - `AnalyticsGUI.displayReport()`
- **Execution:** Upon mission completion, `ReportGenerator.compileMetrics()` aggregates quantitative metrics including rescued counts, battery efficiency, and total ticks directly from `GridEngine`. These stats pass to `GeminiClient`, which synthesizes an executive performance summary displayed inside `AnalyticsGUI`.

3 UML Class Diagram and Design Patterns

3.1 Design Pattern Explanations

3.1.1 1. Command Pattern

- **What design problem does the pattern address?** It decouples the object that invokes or schedules an operation such as the AI planner or operator dashboard from the objects that know how to perform it like the physical robots, enabling request queuing, logging, and undo or redo capabilities.
- **Which classes participate in the pattern?** The `Command` interface and concrete implementations such as `ScanAreaCommand`.
- **What role does each class play?**
  - `Command`: Declares the execution interface including `execute()` and `getRequiredEnergy()`.
  - `ScanAreaCommand`: Implements the interface to encapsulate specific parameters like target bounds and target drones required for an aerial scan task.
- **Why is this pattern appropriate?** In a multi-agent robotic system, commands generated by an LLM or operator must be validated, queued, and prioritized before hitting hardware. The Command pattern turns requests into first-class objects.
- **What would be more difficult if the pattern were not used?** The orchestration layer would need tight, concrete dependencies on every robot type's specific method signatures, making command queuing and safety-gate verification extremely difficult to implement uniformly.

3.1.2 2. Strategy Pattern

- **What design problem does the pattern address?** It enables a family of interchangeable algorithms to be selected and applied dynamically at runtime based on environmental context without altering the classes that use them.
- **Which classes participate in the pattern?** The `SearchStrategy` interface and `LawnmowerScanStrategy` concrete strategy.
- **What role does each class play?**
  - `SearchStrategy`: Defines the common algorithm contract through `computePath()`.
  - `LawnmowerScanStrategy`: Implements systematic grid-sweep path calculation logic.
- **Why is this pattern appropriate?** Different disaster scenarios require different coverage techniques such as grid sweeps, spiral searches, or perimeter checks. Strategy allows algorithms to be plugged into robots interchangeably.
- **What would be more difficult if the pattern were not used?** The codebase would rely on massive conditional blocks inside the robot navigation classes, making it difficult to add new search algorithms cleanly.

3.1.3 3. Composite Pattern

- **What design problem does the pattern address?** It allows clients to treat individual objects and hierarchical compositions of objects uniformly.
- **Which classes participate in the pattern?** `Robot` as the abstract base, `TrackedRover` as the composite container, and `Robodog` as the subunit.
- **What role does each class play?**
  - `Robot`: Establishes the common component interface.
  - `TrackedRover`: Acts as a container holding nested physical subunits.
  - `Robodog`: Represents the leaf or deployable subunit.
- **Why is this pattern appropriate?** It accurately mirrors real-world multi-tier robotic deployments where heavy transport units carry and dispatch autonomous subunits into hazardous zones.
- **What would be more difficult if the pattern were not used?** Managing nested deployments would require custom, non-uniform control loops and separate management classes for parent and child hardware.

3.1.4 4. Observer Pattern

- **What design problem does the pattern address?** It defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically without tight coupling.
- **Which classes participate in the pattern?** The `GridObserver` interface, `GridEngine` subject, and `GridCanvasGUI` observer.
- **What role does each class play?**
  - `GridEngine`: Maintains the simulation grid matrix and notifies subscribers upon state changes.
  - `GridCanvasGUI`: Implements `GridObserver` to redraw the map in real time whenever simulation ticks occur.
- **Why is this pattern appropriate?** It decouples the core simulation engine from the graphical user interface, ensuring backend logic remains independent of frontend rendering.
- **What would be more difficult if the pattern were not used?** The simulation engine would need hardcoded references to UI components, creating severe coupling and rendering synchronization bottlenecks.

3.1.5 5. Decorator and Safety Gate Pattern

- **What design problem does the pattern address?** It intercepts requests dynamically to enforce validation rules, security checks, or operational constraints before execution reaches core components.
- **Which classes participate in the pattern?** `ActionVerifier` working alongside `Command` and `Robot`.
- **What role does each class play?**

  - `ActionVerifier`: Acts as a deterministic validation filter evaluating battery margins and spatial collision risks prior to execution.

- **Why is this pattern appropriate?** Because AI agents can occasionally propose flawed or unsafe plans, a centralized validation gate ensures that all generated commands pass rigorous safety checks before hardware actuation.
- **What would be more difficult if the pattern were not used?** Safety checks would have to be redundantly duplicated inside every individual command and robot class, risking missed edge cases and system failures.

3.1.6 6. Facade Pattern

- **What design problem does the pattern address?** It provides a unified, simplified interface to complex subsystems such as AI clients, tool parsers, and prompt builders, hiding internal complexity from client classes.
- **Which classes participate in the pattern?** `IntelligenceService` and `PlanOrchestrator` acting as facades over external AI clients.
- **What role does each class play?**
  - `IntelligenceService`: Exposes clean, high-level methods for natural language processing and tool invocation.
- **Why is this pattern appropriate?** It shields the rest of the application from intricate boilerplate code required to construct prompts, handle API rate limits, and parse LLM tool outputs.
- **What would be more difficult if the pattern were not used?** User interface and core service classes would be tightly coupled to low-level AI API connection logic and JSON parsing code.

3.1.7 7. Adapter Pattern

- **What design problem does the pattern address?** It allows classes with incompatible interfaces to work together by wrapping an existing class with a compatible target interface.
- **Which classes participate in the pattern?** `IntelligenceFeed` target interface and `TelegramAdapter`.
- **What role does each class play?**
  - `TelegramAdapter`: Translates incoming third-party messaging payloads into standardized internal incident models.
- **Why is this pattern appropriate?** It enables external emergency alert channels like Telegram bots to feed data into the application without forcing changes to core data structures.
- **What would be more difficult if the pattern were not used?** Core ingestion code would need bespoke integration logic for every external messaging platform, violating separation of concerns.

3.1.8 8. Factory Method Pattern

- **What design problem does the pattern address?** It defines an interface for creating objects, but lets specialized factories decide which concrete class to instantiate based on runtime inputs.
- **Which classes participate in the pattern?** `TaskFactory` and `Command`.
- **What role does each class play?**
  - `TaskFactory`: Centralizes instantiation logic for creating specific command objects from unstructured AI task outputs.
- **Why is this pattern appropriate?** Decouples object creation from the orchestration layer when the exact type of command to instantiate depends on dynamic AI evaluation.
- **What would be more difficult if the pattern were not used?** Orchestration classes would be cluttered with complex conditional instantiation logic spread across the codebase.

4 Use-Case Diagram and Descriptions

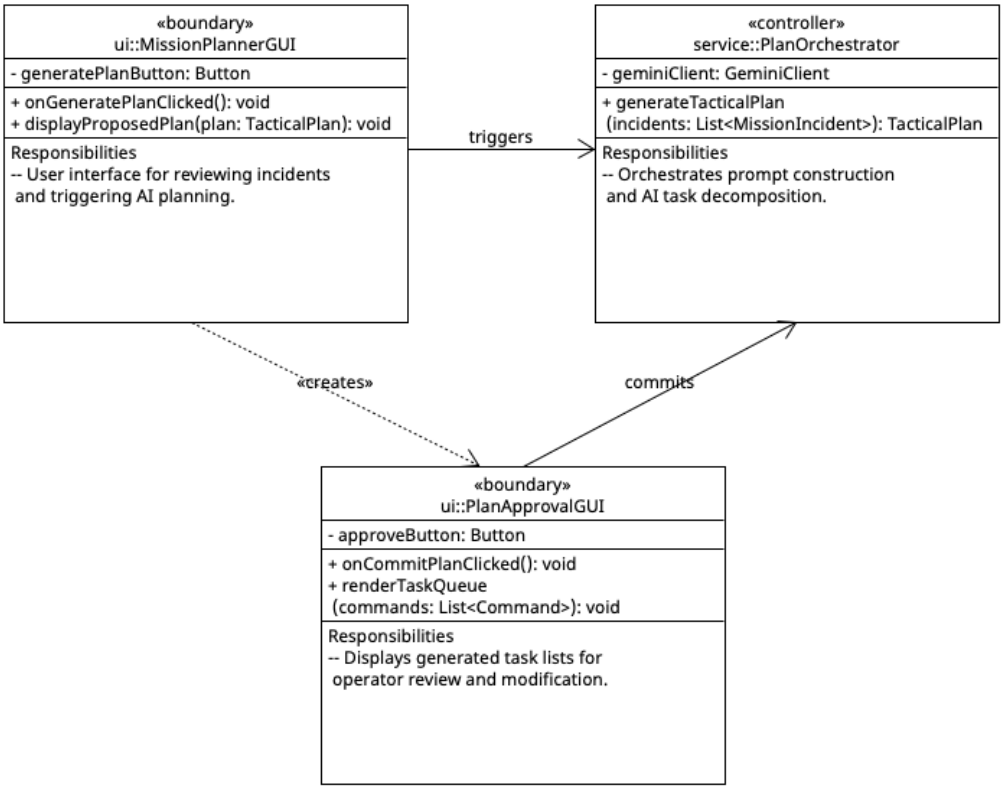


Figure 4: UI and Controller Layer Architecture

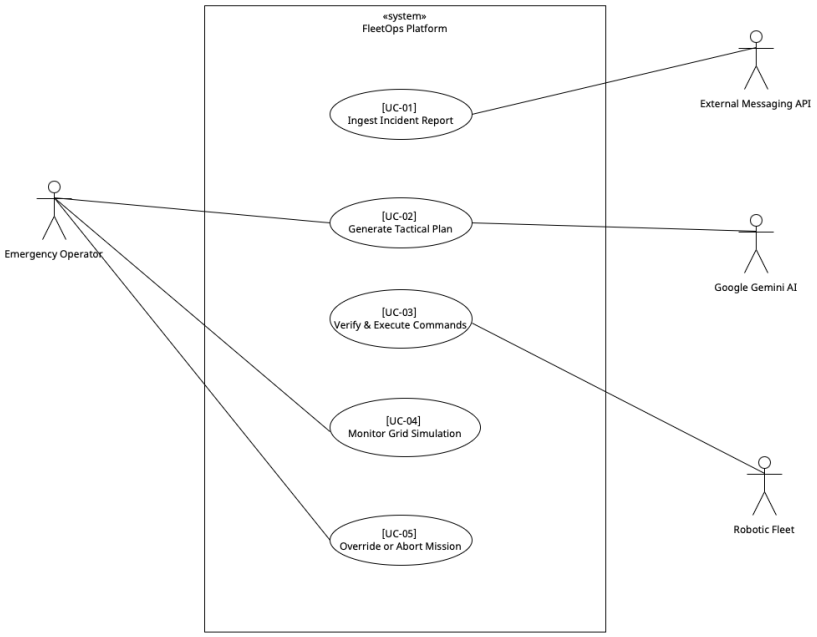


Figure 5: System Use-Case Diagram

4.1 Detailed Use-Case Descriptions

- **UC-01: Ingest Emergency Incident Report**
  - **Use Case ID:** UC-01
  - **Use Case Name:** Ingest Emergency Incident Report
  - **Actor(s):** External Messaging API
  - **Goal:** Automatically parse and ingest incoming third-party disaster telemetry into standard system incident models.
  - **Preconditions:** The messaging gateway and adapter services are active and reachable.
  - **Trigger:** An incoming alert payload is posted to the external webhook endpoint.
  - **Main Success Scenario:**
    1. The external messaging service sends an incident alert payload.
    2. `TelegramAdapter` intercepts the payload and parses the unstructured data.
    3. The adapter maps the data into a standardized internal `MissionIncident` model.
    4. The incident is saved into `DatabaseTool` and queued for tactical evaluation.
  - **Alternative and Exception Flows:**

    2a. *Malformed Payload:* If the incoming payload lacks required spatial or description fields, the system logs a parsing warning and drops the invalid packet.
  - **Postconditions:** A new `MissionIncident` record is persisted and visible on the operator dashboard.
  - **Related Feature(s):** Adapter Pattern and Ingestion Subsystem.
- **UC-02: Generate Autonomous Tactical Plan**
  - **Use Case ID:** UC-02
  - **Use Case Name:** Generate Autonomous Tactical Plan
  - **Actor(s):** Emergency Operator and Google Gemini AI
  - **Goal:** Analyze pending disaster incidents and generate a prioritized sequence of robot commands using AI reasoning.
  - **Preconditions:** At least one unassigned incident exists in the spatial database.
  - **Trigger:** Operator clicks Generate Tactical Plan on `MissionPlannerGUI`.
  - **Main Success Scenario:**
    1. Operator selects target incidents and requests plan generation.
    2. `PlanOrchestrator` fetches relevant context via `DatabaseTool` and constructs the AI prompt.
    3. `GeminiClient` transmits the request to the Google Gemini service.
    4. Google Gemini returns a structured JSON plan containing task decompositions.
    5. `TaskFactory` instantiates concrete `Command` objects.
    6. The proposed tactical plan is rendered on `PlanApprovalGUI` for operator review.
  - **Alternative and Exception Flows:**

    3a. *AI API Timeout or Failure:* If Gemini is unreachable, the system notifies the operator and offers options for manual command assembly.

    5a. *Invalid Task Output:* If the returned JSON fails schema validation, `TaskFactory` throws an exception and requests a single prompt retry.

  - **Postconditions:** A structured tactical plan containing instantiated command objects is staged for validation.
  - **Related Feature(s):** Facade Pattern, Factory Method Pattern, and AI Integration.

- **UC-03: Verify and Execute Robot Commands**
  - **Use Case ID:** UC-03
  - **Use Case Name:** Verify and Execute Robot Commands
  - **Actor(s):** Robotic Fleet
  - **Goal:** Validate proposed commands against safety gates and dispatch them to physical assets for execution.
  - **Preconditions:** A tactical plan has been approved by the operator.
  - **Trigger:** Operator confirms plan deployment on `PlanApprovalGUI`.
  - **Main Success Scenario:**
    1. Operator approves the pending task list.
    2. `ActionVerifier` intercepts each command to check battery reserves and collision boundaries.
    3. Validated commands invoke their `execute()` method.
    4. Commands are transmitted to target units in the robotic fleet.
    5. Robots update their operational states upon task completion.
  - **Alternative and Exception Flows:**

    2a. *Safety Gate Rejection:* If `ActionVerifier` detects inadequate battery, the command is flagged as unsafe and blocked from execution.
  - **Postconditions:** Safe commands are dispatched and hardware state transitions occur in the field.
  - **Related Feature(s):** Decorator Pattern, Command Pattern, and Hardware Interface.

- **UC-04: Monitor Real-Time Grid Simulation**
  - **Use Case ID:** UC-04
  - **Use Case Name:** Monitor Real-Time Grid Simulation
  - **Actor(s):** Emergency Operator
  - **Goal:** Provide continuous visual feedback on field operations and spatial unit locations.
  - **Preconditions:** The spatial grid simulation is running.
  - **Trigger:** The simulation engine completes a tick or unit position update.
  - **Main Success Scenario:**
    1. `GridEngine` updates spatial coordinate matrices and robot locations.
    2. `GridEngine` notifies registered observers of state changes.
    3. `GridCanvasGUI` receives the update event.
    4. The GUI redraws the grid canvas in real time to reflect new robot positions.
  - **Alternative and Exception Flows:**

    3a. *Render Lag:* If user interface thread frame drops occur, updates are batched to preserve simulation performance.

  - **Postconditions:** The visual map display remains synchronized with current simulation telemetry.
  - **Related Feature(s):** Observer Pattern and Grid Simulation Engine.

- **UC-05: Override or Abort Mission**
  - **Use Case ID:** UC-05
  - **Use Case Name:** Override or Abort Mission
  - **Actor(s):** Emergency Operator
  - **Goal:** Allow the operator to immediately halt active hardware operations in an emergency.
  - **Preconditions:** At least one robot is actively executing a mission command.
  - **Trigger:** Operator hits the manual abort button on the dashboard interface.
  - **Main Success Scenario:**
    1. Operator triggers an immediate emergency abort.
    2. System broadcasts a high-priority cancellation command across all active command queues.
    3. Robotic fleet units immediately halt movement and switch to a safe hold state.
    4. The user interface reflects emergency override status.
  - **Alternative and Exception Flows:**

    2a. *Communication Interruption:* If the radio link drops during abort, local unit safety timeouts trigger automatic return-to-base protocols.
  - **Postconditions:** Active execution queues are cleared and physical units transition to safe hold states.
  - **Related Feature(s):** Safety Systems and Manual Override Controls.

5 Sequence Diagrams

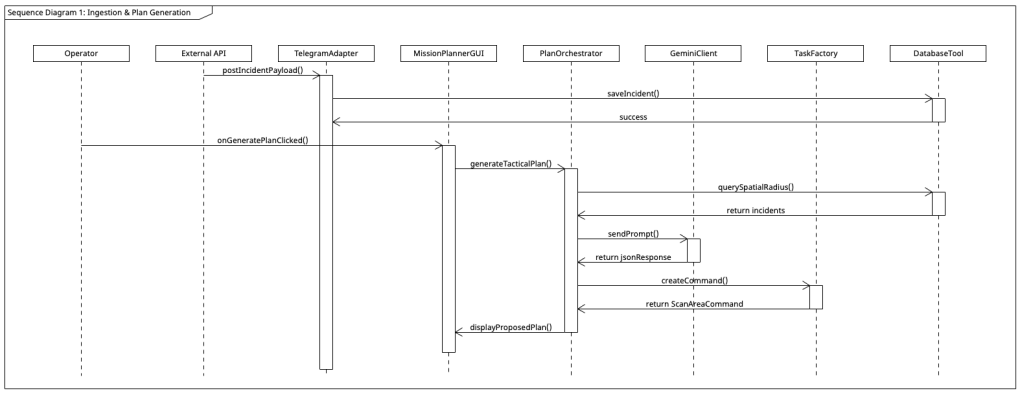


Figure 6: Sequence Diagram for Autonomous Planning and Telemetry Ingestion

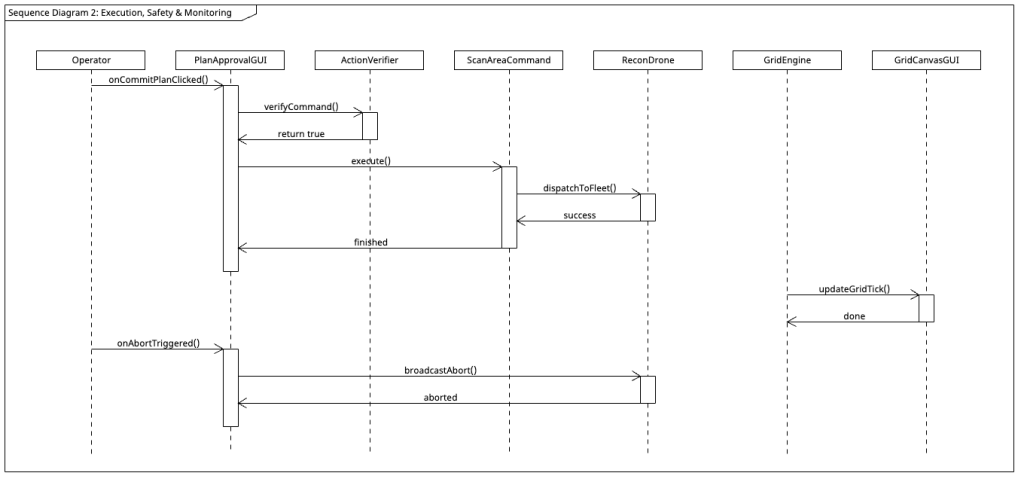


Figure 7: Sequence Diagram for Execution, Safety Verification and Real-Time Monitoring

6 Feature-to-Design Traceability Table

To demonstrate that the proposed software design is concrete and directly supports every planned capability, the table below maps all 10 non-trivial features to their corresponding use cases, participating Java classes, key execution methods, sequence diagrams, and applied design patterns.

Feat.	Description	Type	Use Case	Participating Classes	Key Methods	Seq.	Patterns
F01	Feed Ingestion	AI	UC01	FeedScannerGUI, TelegramAdapter, IntelligenceService, GeminiClient	fetchRawData(), parseFeedText()	SD01	Adapter, Facade
F02	DB Deduplication	Hybrid	UC02	IntelligenceService, DatabaseTool, H2DatabaseManager, VectorSearchEngine	verifyIncident(), queryDatabaseTool()	SD02	Facade, Command
F03	Plan Generation	AI	UC03	MissionPlannerGUI, PlanOrchestrator, GeminiClient, TaskFactory	generatePlan(), createCommand()	SD03	Factory, Facade
F04	Task Approval	Det.	UC04	PlanApprovalGUI, CommandQueueManager	approveTask(), commitToExecution()	SD04	Command, Observer
F05	2D Grid Setup	Det.	UC05	GridCanvasGUI, GridEngine, EntityFactory, ObstacleCell	initializeGrid(), spawnEntity()	SD05	Abstract Factory
F06	Aerial Recon	Hybrid	UC06	ReconDrone, ScanAreaCommand, LawnmowerScanStrategy, ActionVerifier	execute(), validateAction()	SD06	Command, Strategy
F07	Ground Sweep	Hybrid	UC07	TrackedRover, Robodog, TraversePathCommand, RadiusSweepStrategy	execute(), deploySubUnit()	SD07	Command, Composite
F08	Heli Extraction	Hybrid	UC08	RescueHelicopter, EvacuateSurvivorCommand, GridEngine, ActionVerifier	execute(), verifyLandingZone()	SD08	Command, Observer
F09	Safety Gate	Det.	UC09	ActionVerifier, Command, Robot, GridEngine	validateAction(), checkBattery()	SD09	Decorator, Strategy
F10	Analytics Report	AI	UC10	AnalyticsGUI, ReportGenerator, GeminiClient, GridEngine	compileMetrics(), synthesizeReport()	SD10	Facade, Template

Feature-to-Design Traceability Matrix

7 Feature Realization Explanations

1. **F01 — Intelligence Feed Ingestion and Parsing**
   - **Related Use Case:** UC01 — Scan Intel Feed
   - **Related Sequence Diagram:** SD01 — Feed Ingestion and Parsing
   - **Classes Involved:**
     - `FeedScannerGUI` and `FeedScannerCLI`: Captures manual operator text or initiates stream scans.
     - `IntelligenceService`: Orchestrates data fetching and parsing workflows.
     - `TelegramAdapter` and `NewsFeedAdapter`: Converts heterogeneous source feeds into standardized raw feed data.
     - `GeminiClient`: Sends unstructured feed text to Google Gemini for entity extraction.
     - `MissionIncident`: Data structure representing extracted crisis incidents.
   - **Important Methods:**
     - `FeedScannerGUI.onScanButtonClicked()`
     - `IntelligenceService.ingestFeeds()`
     - `TelegramAdapter.fetchData()`
     - `GeminiClient.extractEntities()`
   - **Execution:** When the operator clicks scan feeds in the GUI or issues the scan command in the CLI, `IntelligenceService` invokes `TelegramAdapter.fetchData()`. Raw payloads are passed to `GeminiClient.extractEntities()`, which parses unstructured natural language into structured incident objects that populate the pending intelligence view.

2. **F02 — Vector and Database Deduplication Tool**
   - **Related Use Case:** UC02 — Deduplicate Incidents
   - **Related Sequence Diagram:** SD02 — Database Tool Deduplication Loop
   - **Classes Involved:**
     - `IntelligenceService`: Manages the incident verification lifecycle.
     - `DatabaseTool`: Exposes database search interfaces as an LLM tool.
     - `H2DatabaseManager`: Executes SQL persistence and spatial query operations.
     - `VectorSearchEngine`: Evaluates semantic description similarity.
   - **Important Methods:**
     - `IntelligenceService.verifyIncident()`
     - `DatabaseTool.executeToolCall()`
     - `H2DatabaseManager.querySpatialRadius()`
     - `VectorSearchEngine.findDuplicates()`
   - **Execution:** During feed processing, `GeminiClient` issues a tool call to `DatabaseTool.executeTool`. The tool delegates to `H2DatabaseManager` and `VectorSearchEngine` to perform spatial and semantic similarity checks against previously logged events. If a duplicate exists, the system merges the entries or flags them as resolved.

3. **F03 — Autonomous Heterogeneous Task Plan Generation**
   - **Related Use Case:** UC03 — Generate Tactical Plan

- **Related Sequence Diagram:** SD03 — Plan Generation and Queueing
- **Classes Involved:**
  - `MissionPlannerGUI`: Collects selected target incidents and initiates planning.
  - `PlanOrchestrator`: Coordinates prompt context construction and plan parsing.
  - `GeminiClient`: Generates multi-robot task decomposition strategies.
  - `TaskFactory`: Instantiates concrete command instances from plan specifications.
  - `TacticalPlan`: Encapsulates the generated sequence of robot tasks.
- **Important Methods:**
  - `MissionPlannerGUI.onGeneratePlanClicked()`
  - `PlanOrchestrator.generateTacticalPlan()`
  - `GeminiClient.generate()`
  - `TaskFactory.createCommand()`
- **Execution:** When the user triggers plan generation, `PlanOrchestrator` bundles incident data and active fleet state into a structured prompt. `GeminiClient.generate()` returns a JSON task strategy. `PlanOrchestrator` passes this payload to `TaskFactory.createComma` which instantiates concrete command objects for user review.

4. **F04 — Interactive Task Queue Approval and Manual Override**

- **Related Use Case:** UC04 — Approve and Edit Plan
- **Related Sequence Diagram:** SD04 — Plan Approval and Queueing
- **Classes Involved:**
  - `PlanApprovalGUI` and `PlanApprovalCLI`: Displays generated task lists for modification.
  - `PlanOrchestrator`: Manages queue state adjustments.
  - `CommandQueueManager`: Maintains the active, ordered execution queue.
  - `GridEngine`: Receives approved commands for execution.
- **Important Methods:**
  - `PlanApprovalGUI.onTaskToggled()`
  - `PlanApprovalGUI.onCommitPlanClicked()`
  - `CommandQueueManager.enqueueCommand()`
  - `GridEngine.loadCommandQueue()`
- **Execution:** Proposed tasks are rendered in `PlanApprovalGUI`. The user accepts, rejects, or reorders individual items. Upon clicking commit plan, `CommandQueueManager` validates queue completeness and transfers approved command objects to `GridEngine` for execution.

5. **F05 — 2D Environment and Dynamic Entity Generator**

- **Related Use Case:** UC05 — Initialize 2D Grid
- **Related Sequence Diagram:** SD05 — Grid and Entity Setup
- **Classes Involved:**
  - `GridCanvasGUI`: Renders the interactive 2D matrix map.
  - `GridEngine`: Maintains grid cell occupancy states and tick loops.
  - `EntityFactory`: Instantiates obstacles, terrain cells, and survivors.
  - `SurvivorEntity`: Represents simulated victims with specific movement profiles.

- **Important Methods:**
  - `GridCanvasGUI.initializeCanvas()`
  - `GridEngine.setupEnvironment()`
  - `EntityFactory.createSurvivor()`
  - `SurvivorEntity.updatePosition()`
- **Execution:** The operator selects a terrain preset or manual dimensions. `GridEngine` invokes `EntityFactory` to populate grid cell matrices with terrain hazards and survivor entity instances. `GridCanvasGUI` attaches as an observer to repaint map state changes in real time.

6. **F06 — Aerial Reconnaissance and Path Mapping**
   - **Related Use Case:** UC06 — Execute Recon Scan
   - **Related Sequence Diagram:** SD06 — Aerial Scan Execution
   - **Classes Involved:**
     - `ReconDrone`: Autonomous aerial vehicle subclass.
     - `ScanAreaCommand`: Encapsulates cell coverage flight instructions.
     - `LawnmowerScanStrategy`: Calculates systematic area grid patterns.
     - `ActionVerifier`: Validates flight safety parameters before movement.
     - `GridEngine`: Updates cell visibility and obstacle maps.
   - **Important Methods:**
     - `ScanAreaCommand.execute()`
     - `LawnmowerScanStrategy.computePath()`
     - `ActionVerifier.validateAction()`
     - `GridEngine.clearFogOfWar()`
   - **Execution:** Executing `ScanAreaCommand` triggers `LawnmowerScanStrategy.computePath()` on `ReconDrone`. For each step, `ActionVerifier.validateAction()` checks battery levels and map bounds. Valid moves update cell visibility via `GridEngine.clearFogOfWar()`, establishing safe ground paths for rovers.

7. **F07 — Ground Traversal and Perimeter Radius Sweep**
   - **Related Use Case:** UC07 — Execute Ground Sweep
   - **Related Sequence Diagram:** SD07 — Ground Sweep and Detection
   - **Classes Involved:**
     - `TrackedRover`: Main ground vehicle carrying quadruped assets.
     - `Robodog`: Deployable quadruped subunit.
     - `TraversePathCommand`: Navigates rover along cleared ground vectors.
     - `RadiusSweepStrategy`: Executes circular perimeter scans.
     - `ActionVerifier`: Checks physical movement constraints.
   - **Important Methods:**
     - `TraversePathCommand.execute()`
     - `TrackedRover.deploySubUnit()`
     - `RadiusSweepStrategy.executeSweep()`
     - `ActionVerifier.validateAction()`

- **Execution:** `TraversePathCommand` moves `TrackedRover` along cleared grid coordinates to a designated waypoint. The rover invokes `deploySubUnit()` to dispatch a robodog, which executes `RadiusSweepStrategy` to scan cells within the specified radius. Discovered survivors are flagged on the tactical map.

8. **F08 — Helicopter Extraction Dispatch**
   - **Related Use Case:** UC08 — Dispatch Helicopter
   - **Related Sequence Diagram:** SD08 — Helicopter Extraction
   - **Classes Involved:**
     - `RescueHelicopter`: Heavy-lift extraction flight asset.
     - `EvacuateSurvivorCommand`: Controls extraction flight workflows.
     - `ActionVerifier`: Validates landing zone clearance.
     - `GridEngine`: Updates global survivor rescue counts.
   - **Important Methods:**
     - `EvacuateSurvivorCommand.execute()`
     - `ActionVerifier.checkLandingZone()`
     - `RescueHelicopter.extractTarget()`
     - `GridEngine.markSurvivorRescued()`
   - **Execution:** When a survivor location is confirmed, `EvacuateSurvivorCommand` triggers `RescueHelicopter`. `ActionVerifier.checkLandingZone()` verifies that the destination landing zone is clear of obstacles. Upon landing, the helicopter extracts the survivor and `GridEngine` updates mission metrics.

9. **F09 — Deterministic ActionVerifier Safety Gate**
   - **Related Use Case:** UC09 — Verify Action Feasibility
   - **Related Sequence Diagram:** SD09 — Safety Verification Loop
   - **Classes Involved:**
     - `ActionVerifier`: Central deterministic rule validation engine.
     - `Command`: Base class for executable robot instructions.
     - `Robot`: Base class representing vehicle hardware states.
     - `GridEngine`: Provides terrain and boundary references.
   - **Important Methods:**
     - `ActionVerifier.validateAction()`
     - `ActionVerifier.checkBatteryMargin()`
     - `ActionVerifier.checkObstacleCollision()`
     - `Command.getRequiredEnergy()`
   - **Execution:** Every command execution call is intercepted by `ActionVerifier.validateAction()`. The verifier evaluates `Command.getRequiredEnergy()` against current robot battery levels and queries `GridEngine` for obstacle collisions. Invalid actions are rejected before modifying simulation state.

10. **F10 — Post-Mission Ground-Truth Analytics and Reporting**
    - **Related Use Case:** UC10 — Generate Analytics Report
    - **Related Sequence Diagram:** SD10 — Analytics and Synthesis
    - **Classes Involved:**

  - `AnalyticsGUI`: Displays final mission charts and text reports.
  - `ReportGenerator`: Compiles performance stats and invokes summary templates.
  - `GeminiClient`: Synthesizes qualitative executive performance summaries.
  - `GridEngine`: Provides ground-truth benchmark data.
- **Important Methods:**
  - `ReportGenerator.compileMetrics()`
  - `ReportGenerator.generateReport()`
  - `GeminiClient.synthesizeExecutiveSummary()`
  - `AnalyticsGUI.displayReport()`
- **Execution:** Upon mission completion, `ReportGenerator.compileMetrics()` aggregates quantitative metrics including rescued counts, battery efficiency, and total ticks directly from `GridEngine`. These stats pass to `GeminiClient`, which synthesizes an executive performance summary displayed inside `AnalyticsGUI`.